

Agentic Engineer Capstone — Ops-Ready Runbook

1. Purpose

This runbook provides operational guidance for running, monitoring, troubleshooting, escalating, and recovering the governed multi-agent engineering pipeline.

It is intended for a developer, reviewer, or operator who needs to run the system without relying on undocumented knowledge.

2. System Overview

The workflow coordinates specialized engineering roles through an Orchestrator.

Primary execution path:

text

Developer Change

-> Orchestrator

-> Deterministic Handoff Validation

-> Role Routing

-> Implementer / Reviewer / Tester / Project Manager

-> Policy and Evaluation Gates

-> Human Checkpoint

The architecture combines:

- * agentic reasoning;
- * deterministic validation;
- * MCP-backed storage and retrieval;
- * CI/CD policy checks;
- * human escalation.

3. Prerequisites

Before running the workflow, confirm:

- * Docker is installed and running.
- * The repository is available locally.
- * The Module 4 image is available as `agentic\_engineer\_4:latest`.
- * Required environment variables are loaded.
- * The working tree does not contain unintended secret files.
- * MCP configuration and policy files are present.
- * Evaluation assets are present.

Required or potentially required environment variables include:

```
```text
```

```
OPENROUTER_API_KEY
```

```
SLACK_BOT_TOKEN
```

```
SLACK_TEAM_ID
```

Only provide credentials required for the workflow being executed.

Never hardcode secrets in repository files.

## 4. Pre-Run Verification

From the repository root, run:

```
powershell
```

```
git status
```

Confirm there are no unintended tracked changes.

Then run the local policy suite inside the Docker image:

```
```powershell
```

```
docker run --rm `
```

```
-v "${PWD}:/workspace" `
```

```
-w /workspace `
```

```
agentic_engineer_4:latest `
```

```
pytest eval/test_policy.py -v
```

Expected current policy baseline:

```
text
```

```
6 passed
```

If policy tests fail, stop before running the production-like orchestration.

```
---
```

5. OpenRouter Authentication Check

Confirm that the environment variable is loaded without printing the key:

```
```powershell
```

```
$env:OPENROUTER_API_KEY.Length
```

A positive number indicates the variable is populated.

Do not use:

```
powershell
```

```
echo $env:OPENROUTER_API_KEY
```

because that would expose the secret.

A safe authentication check may query the OpenRouter model endpoint using the bearer token.

A successful response confirms authentication but does not guarantee that every model endpoint is permitted by the managed data p

---

## ## 6. Launching the Container

The course-supported Windows pattern passes the OpenRouter key into the container as an environment variable.

Example:

```
```powershell
docker run -it --rm `
-p 8501:8501 -p 8502:8502 `
-e OPENROUTER_API_KEY=$env:OPENROUTER_API_KEY `
-e SLACK_BOT_TOKEN=$env:SLACK_BOT_TOKEN `
-e SLACK_TEAM_ID=$env:SLACK_TEAM_ID `
-v "${PWD}:/workspace" `
agentic_engineer_4:latest
```

Only include optional credentials if they are actually needed for the run.

7. Production-Like Orchestration Run

The capstone orchestrator is located at:

text

eval/orchestrator.py

The current configured model is:

```
```text
anthropic/claude-sonnet-4.6
```

The orchestrator uses:

text

OPENROUTER\_API\_KEY

and the OpenRouter Anthropic-compatible endpoint.

Run evidence should be written to a repository-mounted path such as:

```
```text
logs/
```

so that transcripts survive the disposable container.

8. Current External Runtime Blocker

During the current capstone run, OpenRouter authentication succeeds and the configured model appears in the model list.

However, the model-backed orchestration currently fails with:

text

404 - No endpoints available matching your guardrail restrictions and data policy

This indicates an endpoint-routing restriction associated with the course-managed OpenRouter policy rather than a missing local endpoint.

Operational response:

- 1\ Do not expose or regenerate the course API key unless instructed.
- 2\ Do not weaken local governance controls to bypass the error.
- 3\ Do not switch models without confirming a course-approved alternative.
- 4\ Preserve the error output as runtime evidence.
- 5\ Escalate the managed policy issue to the course support team.
- 6\ Continue non-runtime capstone work while the external dependency is unresolved.

9. Monitoring

During a workflow run, monitor:

- * agent execution order;
- * handoff validation;
- * routing decisions;
- * MCP tool calls;
- * authorization denials;
- * retrieval classification enforcement;
- * evaluation results;
- * retries;
- * latency;
- * cost;
- * human escalation events.

Relevant evidence sources include:

```text

logs/

.memory/storage-audit.log

eval/

docs/calibration-log.md

docs/retrieval-quality-report.md

## 10. Reliability Controls

The workflow should use or preserve controls such as:

- maximum agent iterations;
- deterministic structural validation;
- policy tests;
- classification ceilings;
- role-specific allow-lists;
- evaluation gates;
- regression tests;
- human escalation;
- bounded retries where implemented.

Do not add unlimited retry loops.

Repeated failure should trigger escalation rather than uncontrolled model calls.

## 11. Permission Failure

### *Symptom*

An agent receives an authorization denial from an MCP server.

### *Response*

- 1\. Identify the role.
- 2\. Identify the attempted operation.
- 3\. Check the corresponding allow-list.
- 4\. Check docs/governance-policy.md.
- 5\. Determine whether the denial matches policy.

If the denial is correct, do not widen access simply to complete the task.

If the role genuinely requires new access, follow the permission-widening process and document the justification.

## 12. Policy Drift

### *Symptom*

Policy documentation and executable configuration disagree.

Example evidence from the tool-evolution drill:

text

```
FAILED eval/test_policy.py::test_documentation_writer_policy_matches_enforcement
Response
```

- 1\. Stop the affected workflow.
- 2\. Identify whether documentation or enforcement is incorrect.
- 3\. Restore the intended policy state.
- 4\. Rerun:

```
```powershell
pytest eval/test_policy.py -v
```

or the Docker equivalent.

- 5\. Confirm the suite returns to a passing state.
- 6\. Record the change and result.

13. Retrieval Classification Failure

Symptom

A role requests material above its classification ceiling.

Response

- 1\ Confirm the role's configured classification ceiling.
- 2\ Confirm the classification of the requested content.
- 3\ Preserve the denial event in the audit evidence.
- 4\ Do not raise the ceiling automatically.
- 5\ Escalate if the task legitimately requires higher-classification material.

A denial can represent correct system behavior rather than a failure.

14. Reviewer or Tester Overreach

Reviewer and Tester roles are intended to remain read-only for tracked project state.

If either role attempts to modify source-controlled work:

- 1\ Stop the action.
 - 2\ Preserve the attempted operation in the audit trail.
 - 3\ Confirm container permissions.
 - 4\ Confirm MCP permissions.
 - 5\ Escalate to the Orchestrator or a human reviewer.
- Do not convert advisory roles into implementation roles during incident handling.

15. Deterministic Validation Failure

Symptom

The handoff validator rejects a handoff.

Response

- 1\ Inspect the handoff structure.
- 2\ Compare it against schemas/handoff.json.
- 3\ Correct the producing step rather than weakening schema validation.
- 4\ Rerun the deterministic validation.
- 5\ Preserve both the failed and corrected evidence if useful for calibration.

16. Evaluation Failure

Symptom

A deterministic or rubric evaluation fails.

Response

- 1\ Identify whether the failure is structural or judgment-based.
- 2\ Do not lower thresholds merely to produce a passing result.
- 3\ Locate the relevant transcript, audit entry, or output.
- 4\ Diagnose the underlying workflow behavior.
- 5\ Make a targeted change.
- 6\ Rerun the evaluation.
- 7\ Record before/after evidence.

17. CI/CD Failure

Symptom

A pull request or workflow fails policy, governed-file, or pipeline-integrity checks.

Response

- 1\.. Open the GitHub Actions failure.
- 2\.. Identify the failing job.
- 3\.. Inspect its artifact or log.
- 4\.. Correct the configuration or code.
- 5\.. Rerun the checks.
- 6\.. Do not use `continue-on-error: true` to silence a required gate.

If a protected-branch rule is bypassed by an authorized user, record that platform-level bypass separately from repository policy enforcement.

18. Human Escalation Criteria

Escalate to a human when:

- a role requests permission beyond its approved policy;
- a role requests data above its classification ceiling;
- a workflow action could modify state outside the intended feature scope;
- deterministic and agentic results conflict materially;
- evaluation results remain below threshold after targeted correction;
- external API policy prevents required course execution;
- rollback would affect shared or persistent project state;
- the system cannot determine task ownership safely.

19. Rollback

Rollback may involve:

- restoring a known-good configuration with Git;
- reverting an allow-list change;
- reverting a policy change;
- reverting deterministic logic;
- reverting a workflow file;
- abandoning an unsafe agent-generated modification.

Before rollback:

- 1\.. preserve relevant failure evidence;
- 2\.. identify the last known-good commit;
- 3\.. confirm the rollback does not remove required governance evidence.

After rollback:

- 1\.. rerun policy tests;
- 2\.. rerun relevant deterministic tests;
- 3\.. rerun regression checks;
- 4\.. confirm the intended state is restored.

20. Incident Evidence

For any meaningful incident, record:

- date and time;
- workflow or task ID;
- affected role;
- attempted operation;
- error or policy result;
- relevant log or transcript;
- action taken;
- escalation decision;
- rollback decision;
- final verification result.

Do not record secrets in incident evidence.

21. Secret Handling

Secrets must never be:

- committed to Git;
- included in README examples as real values;
- pasted into logs;
- shown in walkthrough recordings;
- included in screenshots;
- stored in final PDFs.

Use environment variables and masked or placeholder values.

If a secret is accidentally exposed, stop using it and follow the issuing organization's rotation process.

22. Final Production Verification Checklist

Before calling the capstone workflow production-like and complete, confirm:

- policy tests pass;
- deterministic tests pass;
- required MCP services are available;
- retrieval behavior is validated;
- orchestration completes;
- transcript evidence is saved;
- audit evidence is saved;
- final quality metrics are recorded;
- final latency is recorded;
- final defect-rate proxy or measurement is recorded;
- final cycle time is recorded;
- final cost per run is recorded;
- tool-evolution evidence is preserved;
- no secrets appear in artifacts;
- human checkpoints are demonstrated;
- rollback and escalation paths are documented.

23. Current Operational Status

Local governance, policy, deterministic, and tool-evolution evidence are functioning.

The remaining model-backed production-like run is currently dependent on resolution of the managed OpenRouter guardrail/data-policy restriction.

Until that dependency is resolved, the system should report the runtime block transparently rather than substitute estimated final metrics.

24. Runbook Summary

The operational principle for this system is:

Fail closed, preserve evidence, escalate when authority or data scope is exceeded, and only claim results that were actually measured.